1st laboratory report

From regex to NFA

Author
Flores Cóngora, Paolo Luis

Mentor
Adrián Martínez Manzo

National Autonomous University of Mexico, Computer Science: Compilers

Contents

1 Introduction and Validation ... 3
1.1 Video URL ... 3
1.2 Problem Statement ... 3
1.3 Supported Operators ... 3
1.4 Passing Validation ... 3
2 Implementation Description ... 4
2.1 Data Structures ... 4
2.1.1 NFA Representation ... 4
2.1.2 NFA Fragment Abstraction ... 4
2.1.3 Dynamic Arrays ... 4
2.2 Regex Parser ... 5
2.2.1 Explicit Concatenation Insertion ... 5
2.2.2 Shunting-Yard Algorithm ... 5
2.3 Thompson's Algorithm ... 5
2.3.1 Atom Handler ... 6
2.3.2 Concatenation ... 6
2.3.3 Alternation ... 6
2.3.4 Kleene Star, Positive Closure, and Optional ... 6
2.4 NFA Simulator ... 8
2.4.1 Epsilon Closure Calculation ... 8
2.4.2 Character Processing ... 8
2.5 Design Decisions ... 9
2.5.1 Error Handling Framework ... 9
2.5.2 Memory Management ... 10
3 Results ... 11
3.1 Validation Output ... 11
3.2 Unit Test Coverage ... 11
3.3 Visualization ... 11

1. Introduction and Validation

The goal of this assignment is to create a compiler component that converts regular expressions into their equivalent non-deterministic finite automata (NFA) using Thompson's construction algorithm.

1.1 Video URL

1.2 Problem Statement

Given a regular expression as input, the program must:

1. Parse the expression and convert it to postfix notation
2. Construct an NFA using Thompson's algorithm
3. Simulate the NFA to determine whether multiple input strings are accepted

1.3 Supported Operators

The implementation supports the following regular expression operators:

- | (alternation): matches either left or right operand
- . (concatenation): explicit operator for sequential matching
- * (Kleene star): zero or more repetitions
- + (positive closure): one or more repetitions
- ? (optional): zero or one occurrence

1.4 Passing Validation

The implementation successfully passes all validation tests.

```
[+] up 2/2
 ✔ Image 1_regex_to_nfa-main       Built                                0.5s
 ✔ Container 1_regex_to_nfa-main-1 Recreated                            0.0s
Attaching to main-1
main-1  | -- Configuring done (0.0s)
main-1  | -- Generating done (0.0s)
main-1  | -- Build files have been written to: /app
main-1  | [ 33%] Building C object CMakeFiles/regex_to_nfa.dir/src/main.c.o
main-1  | [ 66%] Building C object CMakeFiles/regex_to_nfa.dir/src/tests.c.o
main-1  | [100%] Linking C executable regex_to_nfa
main-1  | [100%] Built target regex_to_nfa
main-1  | Regex: a|ba*b(a|b)*
main-1  |   postfix ok
main-1  |   nfa ok: 11100
main-1  | Regex: a(b|c)*d
main-1  |   postfix ok
main-1  |   nfa ok: 1110
main-1  | Regex: (a|b)*c
main-1  |   postfix ok
main-1  |   nfa ok: 110
main-1  | Regex: ab
main-1  |   postfix ok
main-1  |   nfa ok: 100
main-1  | Regex: a|b|c
main-1  |   postfix ok
main-1  |   nfa ok: 1110
main-1  | Regex: a*
main-1  |   postfix ok
main-1  |   nfa ok: 1110
main-1  | Regex: ab+c
main-1  |   postfix ok
main-1  |   nfa ok: 110
main-1  | Regex: a?b
main-1  |   postfix ok
main-1  |   nfa ok: 110
main-1  | Regex: (ab)*
main-1  |   postfix ok
main-1  |   nfa ok: 1110
main-1  | Regex: (a|b)c+d
main-1  |   postfix ok
main-1  |   nfa ok: 1110
main-1  |
main-1  | Overall: PASS
main-1 exited with code 0
```

2. Implementation Description

2.1 Data Structures

2.1.1 NFA Representation

The NFA is represented using a state-transition model:

```
typedef struct {
    int id;
    transition trans[MAX_TRANSITIONS];
    int trans_count;
    bool is_accept;
} nfa_node;

typedef struct {
    char on;
    nfa_node* target;
} transition;
```

Each state maintains a fixed-size array of transitions (`MAX_TRANSITIONS = 2`), which is sufficient because:

- Non-epsilon transitions occur only for specific input characters
- Epsilon transitions are limited to structural composition points in Thompson's algorithm
- This design avoids dynamic allocation overhead for common cases

The epsilon transition is represented using a special sentinel value: `#define TRANSITION_EPSILON '\0'`

2.1.2 NFA Fragment Abstraction

Thompson's algorithm operates on fragments during construction:

```
typedef struct {
    nfa_node* start;
    nfa_node* accept;
} nfa_fragment;
```

So each fragment represents a partial NFA with a designated entry and exit point (accepted state), enabling modular composition of larger automata.

2.1.3 Dynamic Arrays

A generic macro-based dynamic array implementation provides type-safe collections without code duplication:

```
#define create_da(TYPE, NAME) \
    typedef struct { TYPE* data; int length; int capacity; } da_##NAME; \
    static inline void da_##NAME##_init(da_##NAME* arr, int init_cap) { \
        arr->data = malloc(sizeof(TYPE) * init_cap); \
        arr->length = 0; \
```

```
        arr->capacity = init_cap; \
    } \
    // ... append, pop, free functions
```

This approach declares all necessary functions (`da_T_init`, `da_T_append`, `da_T_pop`, `da_T_free`) for each type without manual repetition.

2.2 Regex Parser

2.2.1 Explicit Concatenation Insertion

Our syntax allows the user to omit the concatenation operator, writing `ab` instead of `a.b`. The parser's first pass makes these implicit operators explicit:

```
int prev_ends = is_operand(prev) || prev == ')' || prev == '*' ||
                prev == '+' || prev == '?';
int curr_starts = is_operand(c) || c == '(';

if (prev_ends && curr_starts) {
    da_char_append(&infix, '.');
}
```

The insertion occurs between:
- Any "closing" element (operand, '`)`' or postfix operator)
- Any "opening" element (operand or '`(`')

For example: `ab+c?d` becomes `a.b+.c?.d`

2.2.2 Shunting-Yard Algorithm

The second pass converts infix notation to postfix using the standard shunting-yard algorithm with left-associative operators:

```
if (top_prec >= curr_prec) {
    da_char_append(&output, *da_char_pop(&op_stack));
} else {
    break;
}
```

Operator precedence is defined as:
- `|` (alternation): precedence 1
- `.` (concatenation): precedence 2
- *, +, ? (postfix): precedence 4

The use of >= (rather than >) ensures left-associative evaluation, meaning operators of equal precedence are processed in the order they appear.

2.3 Thompson's Algorithm

Thompson's algorithm constructs NFAs through composition of primitive fragments. The implementation provides four fundamental operations:

2.3.1 Atom Handler

Creates a single-state NFA for a character:

```
nfa_fragment create_atom(char c) {
    nfa_fragment frag = {0};
    frag.start = create_nfa_node();
    frag.accept = create_nfa_node();
    add_transition(frag.start, frag.accept, c);
    return frag;
}
```

2.3.2 Concatenation

Connects two fragments sequentially via epsilon transition:

```
nfa_fragment concat(nfa_fragment a, nfa_fragment b) {
    a.accept->is_accept = false;
    add_transition(a.accept, b.start, TRANSITION_EPSILON);
    return (nfa_fragment){ .start = a.start, .accept = b.accept };
}
```

2.3.3 Alternation

Creates parallel paths:

```
nfa_fragment union_nfa(nfa_fragment a, nfa_fragment b) {
    nfa_fragment result = {
        .start = create_nfa_node(),
        .accept = create_nfa_node()
    };
    add_transition(result.start, a.start, TRANSITION_EPSILON);
    add_transition(result.start, b.start, TRANSITION_EPSILON);
    add_transition(a.accept, result.accept, TRANSITION_EPSILON);
    add_transition(b.accept, result.accept, TRANSITION_EPSILON);
    return result;
}
```

2.3.4 Kleene Star, Positive Closure, and Optional

The Kleene star creates a loop with a bypass path:

```
nfa_fragment kleene(nfa_fragment a) {
    nfa_fragment result = {
        .start = create_nfa_node(),
        .accept = create_nfa_node()
    };
    add_transition(result.start, a.start, TRANSITION_EPSILON);
    add_transition(result.start, result.accept, TRANSITION_EPSILON);
    add_transition(a.accept, a.start, TRANSITION_EPSILON);
    add_transition(a.accept, result.accept, TRANSITION_EPSILON);
    return result;
}
```

The + operator is equivalent to `a.a*`.

```
nfa_fragment plus(nfa_fragment a) {
    nfa_fragment result = { .start = a.start, .accept = create_nfa_node() };
    add_transition(a.accept, a.start, TRANSITION_EPSILON);
    add_transition(a.accept, result.accept, TRANSITION_EPSILON);
    return result;
}
```

The optional operator is equivalent to `a | epsilon`, requiring two paths to the accepted state:

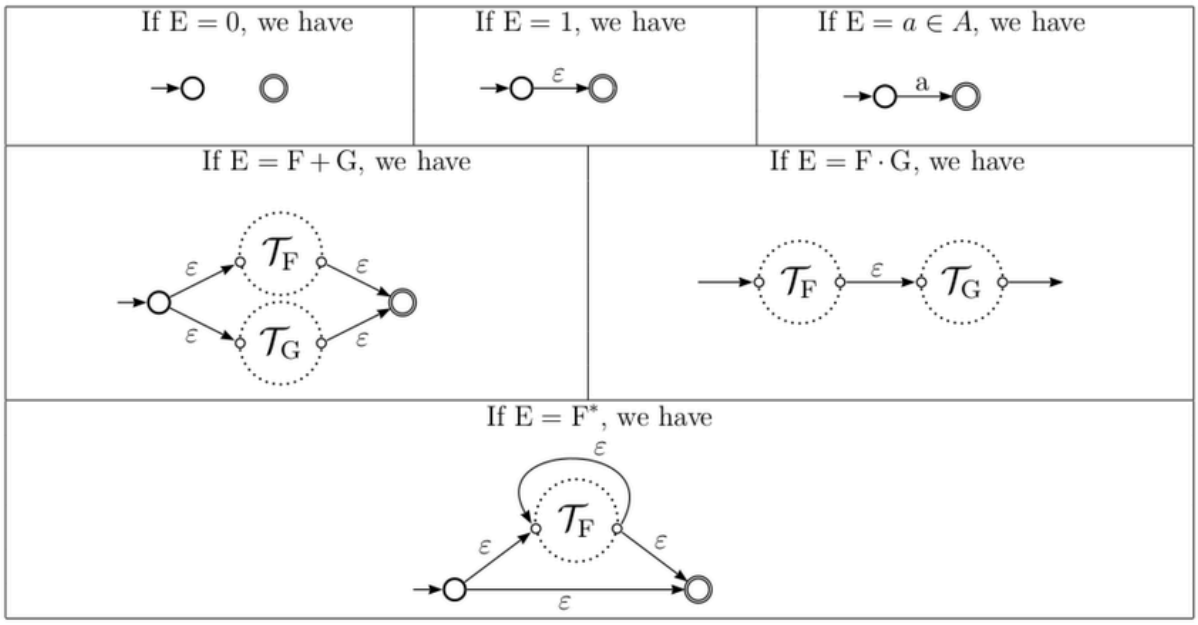
```
nfa_fragment question(nfa_fragment a) {
    nfa_fragment result = {
        .start = create_nfa_node(),
        .accept = create_nfa_node()
    };
    add_transition(result.start, a.start, TRANSITION_EPSILON);
    add_transition(a.accept, result.accept, TRANSITION_EPSILON);
    add_transition(result.start, result.accept, TRANSITION_EPSILON);
    return result;
}
```

Construction from postfix notation uses a stack of fragments, applying the appropriate operation for each token:

```
for (int i = 0; i < r.size; i++) {
    char token = r.items[i].value;

    if (is_operand(token)) {
        da_nfa_fragment_append(&stack, create_atom(token));
    } else if (token == '|') {
        nfa_fragment b = *da_nfa_fragment_pop(&stack);
        nfa_fragment a = *da_nfa_fragment_pop(&stack);
        da_nfa_fragment_append(&stack, union_nfa(a, b));
    } // ... handle '.', '*', '+', '?'
}
```

The following image shows a visual representation of the fragments and how they can be composed.



2.4 NFA Simulator

The simulator determines whether a given string is accepted by the constructed NFA. It operates by tracking all simultaneously active states, accounting for epsilon transitions through epsilon closure computation.

2.4.1 Epsilon Closure Calculation

A recursive helper function computes the epsilon closure of a state:

```
void add_state_with_closure(nfa_node* node, nfa_node** states, int* count) {
    if (*count >= MAX_STATES) return;

    // Check if already in states
    for (int i = 0; i < *count; i++) {
        if (states[i] == node) return;
    }

    states[(*count)++] = node;

    // Follow epsilon transitions
    for (int i = 0; i < node->trans_count; i++) {
        if (node->trans[i].on == TRANSITION_EPSILON) {
            add_state_with_closure(node->trans[i].target, states, count);
        }
    }
}
```

The algorithm prevents infinite loops by maintaining a set of visited states and limits the total number of active states with a fixed array `nfa_node* current_states[MAX_STATES]`.

2.4.2 Character Processing

For each input character, the simulator:

1. Computes which states can be reached from the current set via that character

2. For each subsequent state, computes its epsilon closure
3. Updates the current state set to these new states

```
for (int i = 0; i < buflen; i++) {
    char c = buf[i];
    next_count = 0;

    for (int j = 0; j < current_count; j++) {
        nfa_node* state = current_states[j];
        for (int k = 0; k < state->trans_count; k++) {
            if (state->trans[k].on == c) {
                add_state_with_closure(state->trans[k].target, next_states,
&next_count);
            }
        }
    }
    for (int j = 0; j < next_count; j++) {
        current_states[j] = next_states[j];
    }
    current_count = next_count;
}
```

Acceptance is determined by checking whether any final state in the current set is marked as accepting.

2.5 Design Decisions

2.5.1 Error Handling Framework

The implementation uses a result monad pattern with optional format messages:

```
static inline result(regex) parse_regex(const char* str) {
  result(regex) res = {0};

  if (!str || *str == '\0') {
      res.err = create_error("Empty regex string");
      return res;
  }

  // continue...
}

// The Result syntax

result(regex) rgx = parse_regex(regex_str);
must(rgx, "Could not parse expression '%s'", regex_str);
```

```
❯ cmake . && make && (echo '?|?.???asd.' | ./regex_to_nfa -r)

-- Configuring done (0.0s)
-- Generating done (0.0s)
-- Build files have been written to: /home/paolo/Dev/unam/compilers/repo_lab/1_regex_to_nfa
[ 33%] Building C object CMakeFiles/regex_to_nfa.dir/src/main.c.o
[ 66%] Building C object CMakeFiles/regex_to_nfa.dir/src/tests.c.o
[100%] Linking C executable regex_to_nfa
[100%] Built target regex_to_nfa
result.h:111(must_regex): must() failed: Could not parse expression '?|?.???asd.': Malformed expression
```

And thanks to this pattern, we can easily write asserts with pretty formatted debugging information and type-safe error checking, altought I had to use tricky/advanced macro features that may be difficult to extend.

2.5.2 Memory Management

Manual management was avoided as much as possible to make the implementation easier to read. Multiple parts of the code prefer to allocate fixed-size arrays in the stack, and making assumptions that the input will not exceed predefined limits.

3. Results

3.1 Validation Output

As shown before, the implementation passes all the `validator` tests.

```
[+] up 2/2
 ✔ Image 1_regex_to_nfa-main       Built                                0.5s
 ✔ Container 1_regex_to_nfa-main-1 Recreated                            0.0s
Attaching to main-1
main-1  | -- Configuring done (0.0s)
main-1  | -- Generating done (0.0s)
main-1  | -- Build files have been written to: /app
main-1  | [ 33%] Building C object CMakeFiles/regex_to_nfa.dir/src/main.c.o
main-1  | [ 66%] Building C object CMakeFiles/regex_to_nfa.dir/src/tests.c.o
main-1  | [100%] Linking C executable regex_to_nfa
main-1  | [100%] Built target regex_to_nfa
main-1  | Regex: a|ba*b(a|b)*
main-1  |   postfix ok
main-1  |   nfa ok: 11100
main-1  | Regex: a(b|c)*d
main-1  |   postfix ok
main-1  |   nfa ok: 1110
main-1  | Regex: (a|b)*c
main-1  |   postfix ok
main-1  |   nfa ok: 110
main-1  | Regex: ab
main-1  |   postfix ok
main-1  |   nfa ok: 100
main-1  | Regex: a|b|c
main-1  |   postfix ok
main-1  |   nfa ok: 1110
main-1  | Regex: a*
main-1  |   postfix ok
main-1  |   nfa ok: 1110
main-1  | Regex: ab+c
main-1  |   postfix ok
main-1  |   nfa ok: 110
main-1  | Regex: a?b
main-1  |   postfix ok
main-1  |   nfa ok: 110
main-1  | Regex: (ab)*
main-1  |   postfix ok
main-1  |   nfa ok: 1110
main-1  | Regex: (a|b)c+d
main-1  |   postfix ok
main-1  |   nfa ok: 1110
main-1  |
main-1  | Overall: PASS
main-1 exited with code 0
```

3.2 Unit Test Coverage

The implementation includes 73+ test conditions available in the `tests.c` compilation unit. The `-x` flag can be used to run the test suite.

After uploading this report, I'm planning to make the implementation more robust, so more complex tests are on the way.

3.3 Visualization

I didn't had enough time to achieve this.

I am planning on porting this code to WASM and make the entire compiler available in my personal web page.

For the visualization engine, instead of relying in complex canvas integration, I will simply render the graphs using SVG format.

I have already tested this approach for a logic expressions evaluator that I wrote in Rust and then ported to WASM: https://paoloose.site/discmaths/project/1/

I have not decided yet if I want to take upon this work, but if I do so, I'll be linking the URL in the private comments section :)